Syntactic cues or semantic constraints? a split that demystifies LLM type inference and nondeterminism.

Anonymous Authors[1]

Abstract

Large language models (LLMs) can produce different answers across repeated generations and meaning-preserving perturbations, but such instability is typically measured in aggregate. We instead ask what makes an individual prediction stable. Using Python type inference as a controlled setting, we introduce *semantic inferability*, an input-side characterization of how strongly program constraints determine a prediction independently of the LLM output. Across 52,734 annotation sites and six LLMs, fully constrained sites exceed sites with no detected hard type constraint in stable-informative yield by 14.18–37.64 percentage points. Structure-preserving identifier renaming widens this separation to 29.49–67.90 points: renaming reduces stable-informative yield 15.31–34.99 points more at unconstrained than at fully constrained sites. Among predictions that are initially stable and informative, sites with no detected hard constraint incur 5.29–32.35 points more information loss after renaming. Within individual structured annotations, constraint-supported components are preserved in 97.34–100.00% of paired cases, compared with 56.28–76.36% for residual components. The direction is shared across all six model families, while its magnitude differs significantly across models. These results support a *constraint-guided completion* account: hard semantic constraints form a robust core, while weaker structure, lexical cues, and learned priors complete the residual inference problem.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

1. Introduction

Large language models (LLMs) are remarkably capable, yet their predictions can be surprisingly unstable. Repeating the same request can produce different answers even when the input and model configuration are unchanged (Ouyang et al., 2025; Atil et al., 2024), and predictions can change substantially under seemingly innocuous or meaning-preserving perturbations (Sclar et al., 2024). Repeated sampling has therefore become a common way to expose uncertainty and disagreement in language-model reasoning (Wang et al., 2023; Kuhn et al., 2023). These observations establish an important practical problem, but leave open a more basic interpretive question: *what makes an individual prediction stable?*

Intuitively, information strongly constrained by other information in the input should be harder to change than information that the input leaves underdetermined. If several relationships constrain the same conclusion, a model has less freedom in completing the answer. When the input leaves more alternatives open, learned associations and contextual cues can play a larger role. This suggests a more fine-grained question than whether an LLM is stable in aggregate:

> *Does the stability and robustness of an individual prediction depend on how strongly the input itself constrains that prediction?*

Observed model behavior alone cannot answer this question because the same output may arise from very different sources of information. Recent work on language-model memorization highlights an analogous attribution problem: reproducing a datapoint does not by itself distinguish memorization from generalization (Morris et al., 2026). We study an orthogonal attribution problem at inference time: when an LLM repeatedly produces the same answer, how much of that answer is already determined by the current input, and how much must be supplied from other evidence?

We investigate this question through *LLM-based type inference for Python*. Type inference is particularly useful because program relationships constrain different annotation sites to very different degrees. Calls, assignments,

returns, constructors, attributes, containers, and dependency signatures can completely determine some types, partially determine others, or leave substantial freedom. Learned type-inference systems have long benefited from combining such structural relationships with lexical information (Wei et al., 2020; Allamanis et al., 2020; Wei et al., 2023; Peng et al., 2023), making type inference a natural setting in which to separate these sources of support. It also provides a controlled intervention: lexical and API identity can be removed through structure-preserving renaming while binding identity and program relationships are retained.

Figure 1 illustrates the central phenomenon with two parameters named `hass` from the same source file. GPT-5 predicts `HomeAssistant` for both. At one site, a dependency call constrains the parameter to `HomeAssistant`; at the other, our analysis finds no hard constraint determining that nominal type. After structure-preserving identifier renaming, the first prediction survives under the consistently renamed type identity, whereas the second falls back to `Any`, Python's unconstrained dynamic type. Thus, identical and initially stable predictions can have different evidential foundations. This example illustrates one possible consequence of removing lexical identity rather than a universal outcome for `none` sites: other models can preserve the same prediction by exploiting weaker renaming-invariant evidence not captured by the hard-constraint analysis.

We formalize this distinction through *semantic inferability*, which characterizes how strongly the input determines an annotation independently of the LLM prediction. Rather than using inferability to build another type predictor, we use it as an *interpretive instrument*. We evaluate 52,734 annotation sites from 500 Python files with six LLMs—GPT-5, GPT-6, Claude, DeepSeek, Qwen3-Coder, and Devstral 2—using four repeated generations on the original programs and four after structure-preserving identifier renaming. Repetition measures whether useful information stabilizes when evidence is fixed; renaming removes project-specific lexical identity while preserving program relationships.

Our results reveal a consistent but nonuniform relationship between inferability and model behavior. On the original programs, sites fully determined by hard constraints exceed sites with no detected hard type constraint in stable-informative yield by 14.18–37.64 percentage points. Removing lexical identity amplifies the separation: the corresponding gaps increase to 29.49–67.90 points. Equivalently, iso-renaming reduces stable-informative yield 15.31–34.99 points more at `none` sites than at concrete sites, and every model-specific effect remains positive under project-level resampling.

The same distinction appears after conditioning on predictions that have already stabilized. Relative to concrete sites, `none` sites incur 5.29–32.35 percentage points more total information loss after renaming. The effect persists separately for parameters and returns and under project-level resampling. More locally, at shape-only sites whose original prediction is semantically consistent with the inferred constraint, constraint-supported components survive renaming in 97.34–100.00% of paired annotations, compared with only 56.28–76.36% for residual components of those same annotations.

At the same time, the size of these effects differs across model families. Project-clustered interaction tests detect significant model heterogeneity, and the identity of failed unconstrained sites overlaps only partially across models. Hard semantic constraints therefore do not make model families identical. Instead, they provide a common robustness anchor; when those constraints leave a residual inference problem, differences in weaker structural evidence, learned associations, and model capability become substantially more visible.

These results support a *constraint-guided completion* account. Hard semantic constraints anchor predictions and restrict the space of possible completions; weaker renaming-invariant structural evidence can narrow the remaining alternatives; and lexical/API cues and learned priors supply residual specificity where the input leaves freedom. Under this view, apparent LLM nondeterminism is not uniform noise: it is substantially structured by how strongly the available information constrains what can be generated.

This paper makes three contributions:

- **Semantic inferability as an input-side explanatory framework.** We introduce semantic inferability to characterize how strongly program relationships determine a requested type independently of an LLM prediction, separating a property of the prediction problem from behavior observed only after generation.
- **Controlled evidence that semantic support forms a robust core.** Across six LLMs, structure-preserving lexical removal disproportionately harms sites without hard constraints. The effect persists under parameter/return controls, project-level resampling, common-site comparisons, and a within-annotation analysis in which constraint-supported components are substantially more robust than residual components.
- **Constraint-induced robustness and residual-evidence divergence.** The inferability effect has the same direction across all six model families but differs significantly in magnitude. Hard constraints provide a shared robustness anchor, whereas predictions left to residual inference exhibit substantially greater model-dependent variation in both robustness and failure identity.

(a) Concrete: dependency-constrained

Original program

```
async def async_setup(hass, config):
    ...
    hls_endpoint = async_setup_hls(hass)
    ...
```

Dependency slice

```
def async_setup_hls(
    hass: HomeAssistant) -> str: ...
```

`hass` $\xrightarrow{\text{actual-to-formal}}$ `HomeAssistant`

GPT-5: `hass: HomeAssistant`

After structure-preserving renaming

```
async def f_1571(v_1416, v_0215):
    ...
    v_1702 = xf_0532(v_1416)
    ...
```

Renamed dependency slice

```
def xf_0532(
    v_1502: C_1715) -> str: ...
```

`v_1416` $\xrightarrow{\text{actual-to-formal}}$ `C_1715`

GPT-5: `v_1416: C_1715`

The constraining relationship survives renaming.

(b) No hard nominal constraint

Original program

```
async def check_stream(
    hass, source, pyav_options=None):
    await hass.loop.run_in_executor(
        None, _check_stream_client_error,
        hass, source, pyav_options)
```

`hass` $\dashrightarrow$ `HomeAssistant`

Hard constraint: none determining `HomeAssistant`

GPT-5: `hass: HomeAssistant`

`hass` provides a strong project-specific cue.

After structure-preserving renaming

```
async def f_1511(
    v_0118, v_0359, v_0426=None):
    await v_0118.a_0406.a_1346(
        None, f_0894,
        v_0118, v_0359, v_0426)
```

`hass` $\mapsto$ `v_0118`

GPT-5: `v_0118: Any`

Removing lexical identity removes the `HomeAssistant` prediction.

`async_setup.hass` $\mapsto$ `v_1416`, `check_stream.hass` $\mapsto$ `v_0118`, `HomeAssistant` $\mapsto$ `C_1715`.

Figure 1. **Same prediction, different evidential support.** For readability, `check_stream` abbreviates the original source function `async_check_stream_client_error`. GPT-5 initially predicts `HomeAssistant` for two parameters named `hass` in the same source file. In `async_setup` (a), passing `hass` to `async_setup_hls`, whose corresponding dependency formal is typed `HomeAssistant`, concretely determines the parameter type. Structure-preserving renaming retains this actual-to-formal relationship, and GPT-5 correspondingly predicts the renamed type `C_1715`. In `check_stream` (b), no hard constraint determines the nominal type `HomeAssistant`. GPT-5 nevertheless predicts it in the original program, where `hass` provides a strong project-specific lexical cue. After that binding is independently renamed to `v_0118`, GPT-5 instead predicts `Any`. Distinct local bindings are renamed independently, while dependency relationships and type identities are preserved.

2. From Stability to Inferability

A stable prediction need not have a stable *reason*. Two outputs may be identical even when one is strongly determined by the input and the other depends primarily on residual evidence. Figure 1 illustrates this distinction using two parameters from the same real-world Python source file. We use `check_stream` throughout the paper as a short name for the original source function shown in the figure.

Both parameters are named `hass`, and GPT-5 predicts `HomeAssistant` for both. In `async_setup`, `hass` is passed to `async_setup_hls`; the dependency slice contains `async_setup_hls(hass:HomeAssistant) -> str`, so the call induces an actual-to-formal constraint that determines the local parameter type. In `check_stream`, by contrast, the complete function body contains no corresponding hard constraint that determines the nominal type `HomeAssistant`. The identifier `hass`, however, provides a strong project-specific lexical cue.

The distinction becomes visible after structure-preserving renaming. The two independent bindings receive different local names: `async_setup.hass` becomes `v_1416`, whereas `check_stream.hass` becomes `v_0118`. Dependency relationships and type identities are preserved consistently, with `HomeAssistant` renamed to `C_1715`. The first site's actual-to-formal relationship therefore survives, and GPT-5 continues to predict `C_1715`; at the second site, GPT-5 instead predicts `Any`. Thus, the example exposes the central identification problem: the same prediction need not have the same evidential support. The GPT-5 failure at the second site is illustrative rather than universal: `none` means that no useful hard constraint is established, not that no weaker renaming-invariant evidence remains for another model to exploit.

We organize the evaluation around three questions:

RQ1: Inferability and stabilization. Does stronger semantic inferability make useful predictions more likely to stabilize across repeated generations?

RQ2: Inferability and robustness. Does stronger inferability make established type information more robust when lexical and API identities are removed?

RQ3: Inferability and model variation. Is the relationship between semantic inferability and robustness shared across model families, and how does its magnitude vary across models?

3. Semantic Inferability and Controlled Renaming

3.1. Semantic Inferability

For each target annotation site, we analyze exactly the unannotated source program and dependency context supplied to the LLM. The analysis extracts hard typing relationships induced by assignments, calls, returns, constructors, attributes, containers, dependency signatures, and other supported program relationships, and propagates them to determine what type information can be established independently of the model prediction.

For example, in Fig. 1, `async_setup_hls(hass)` connects the local argument `hass` to a dependency formal typed `HomeAssistant`. This relationship determines the local parameter type without relying on the lexical spelling `hass`.

We summarize the resulting solution space using four inferability classes:

***Concrete*.** Hard constraints determine a complete type, with no unresolved structural component.

***Partial*.** Substantive type components are determined while other components remain unresolved. For example, `dict[str,?]` fixes the dictionary structure and key type but not the value type.

***Shape-only*.** The constraints primarily determine an outer type structure while leaving its substantive element types unresolved.

***None*.** The analysis derives no useful hard structural type reference for the site.

The classes characterize what our analysis can establish rather than all information that could in principle be extracted from Python. Dynamic attribute access, reflection, higher-order behavior, mutable state, incomplete dependency annotations, and other dynamic features make complete recovery of all semantically relevant typing information infeasible in general. We therefore deliberately use a conservative analysis. In particular, `none` means that no useful hard type constraint is established; it does not imply that weaker structural evidence is absent.

This distinction is important for our interpretation. Hard constraints provide a model-independent source of evidence and can anchor a robust core of the prediction. Additional specificity beyond those constraints must be supplied from residual evidence, including weaker structure, lexical/API cues, or learned priors.

3.2. Structure-Preserving Identifier Renaming

Inferability characterizes the hard information present in the input, but does not identify which residual evidence a model uses. We therefore construct a renamed version of each program that removes project-specific identifier and API identities while preserving the program relationships relevant to the analysis.

Renaming is binding-sensitive: each local binding is renamed independently and all uses of that binding are renamed consistently. Project-level type identities and dependency references are also renamed consistently. Thus, binding identity and program structure are preserved while lexical identity is removed.

This transformation separates two sources of support that coexist in the original program. Relationships such as actual-to-formal call constraints survive renaming, whereas lexical correspondences between identifiers such as `hass` and types such as `HomeAssistant` do not.

We interpret the intervention conservatively. Information preserved after renaming has a renaming-invariant source of support. Information that disappears depended on evidence removed by the transformation, but we do not claim that every lost prediction was based exclusively on identifier names.

4. Experimental Design

4.1. Corpus

We construct the evaluation corpus from ManyTypes4Py (Mir et al., 2021). Starting from a previously sampled pool of 2,550 files spanning 50 projects, we select the 500 files containing the largest numbers of parameter sites. The resulting 500-file evaluation corpus spans 36 projects and provides a large population of parameter annotations together with the surrounding source and dependency context needed for inferability analysis.

Across these files, the analysis considers 52,734 parameter and return annotation sites. Their inferability distribution is

Concrete	21,518
Partial	467
Shape-only	991
None	29,758.

4.2. Models and Output Coverage

We evaluate six LLMs spanning proprietary and open-weight model families: GPT-5, GPT-6, Claude, DeepSeek, Qwen3-Coder, and Devstral 2. For each model, we request four annotation generations on the original program and four on its renamed counterpart.

All six models are evaluated on the same 52,734-site source corpus. GPT-5, GPT-6, Claude, and DeepSeek produce usable outputs for the full population, whereas Qwen3-Coder and Devstral 2 cover 42,997 and 36,465 sites, respectively. Their reduced coverage results from file-level generation failures—for example, replacing the requested annotated program with natural-language explanation or producing severely malformed or incomplete code—rather than from different corpus selection. We exclude outputs that cannot be reliably repaired and aligned to the source; model-specific rates therefore use each model's usable population, while site-by-site cross-model analyses use the common population.

4.3. Stable-Informative Yield

Repeated generation measures whether useful information stabilizes while the available evidence remains fixed. A site is *stable* when all four runs agree on the same annotation. Raw agreement alone is insufficient, however: four predictions of root `Any` or `object` are stable but carry little useful type information.

We therefore call a site *stable-informative* when all four generations agree on an informative type. Bare `Any` and bare `object` are treated as low-information roots, whereas structured types that retain useful shape remain informative even when some components are unresolved. Thus, `list[Any]` is informative.

For inferability class C, stable-informative yield is

$$SI_C = \frac{\#\{\text{stable-informative sites in } C\}}{\#\{\text{all evaluated sites in } C\}}.$$

The denominator includes unstable, missing, and low-information outcomes, so SI_C measures the unconditional yield of useful stable information.

4.4. Information Loss Under Renaming

Stable-informative yield asks whether useful information repeatedly emerges. Renaming asks whether information that has already stabilized survives removal of lexical identity.

We therefore condition this analysis on sites that are stable-informative in the original program. We classify the renamed outcome relative to the original stable type. Changing `list[str]` to `list[Any]`, for example, constitutes partial information loss, whereas changing `HomeAssistant` to root `Any` or consistently omitting the annotation constitutes complete loss. We use *total information loss* for the union of partial and complete loss.

Renamed predictions that become unstable are retained as a separate outcome rather than silently classified as information loss. This distinction lets us separate loss of established type information from loss of repeated-run stability. As a sensitivity analysis, we additionally treat root `object` as equivalent to `Any`; this leaves all aggregate denominators and reported loss rates unchanged across the six models.

4.5. Common-Site and Site-Kind Controls

Two selection effects require explicit controls. First, model output coverage differs, especially for Qwen3-Coder and Devstral 2. We therefore report both model-specific populations and analyses restricted to sites shared across model families. For information-loss comparisons, the strict common population contains sites at which every supplied model made an originally stable-informative prediction. This ensures that cross-model differences are not merely consequences of different model-specific denominators.

Second, inferability classes have very different annotation-site composition. Of the 21,518 concrete sites, 16,029 (74.49%) are returns. In contrast, only 3,497 of the 29,758 `none` sites (11.75%) are returns. We therefore supplement whole-population results with comparisons performed separately for parameters and returns.

4.6. Component-Level Analysis

The extreme inferability classes ask whether a site as a whole is determined by hard constraints. Partial and shape-only sites support a more localized test: within the same structured annotation, some components are supported by hard program relationships while others remain unresolved.

We therefore project predictions at these sites into *constraint-supported* and *residual* components and compare their behavior. The supported projection contains type information established by the hard constraints; the residual projection contains components left unresolved by those constraints. We measure both within-condition stability and original-to-renamed preservation.

The component analysis begins from 1,155 partial and shape-only target sites common across all six models. Predictions must contain localizable component paths; cross-condition comparisons additionally require unique supported and residual projection modes in both the original and renamed programs. Missing or unlocalizable cases are excluded rather than counted as instability. Because the supported and residual projections come from the same annotation, this analysis controls for site-level differences that arise when comparing separate inferability classes.

Our primary mechanism analysis focuses on shape-only sites whose *original* modal prediction is semantically consistent with the constraint-derived structure. Selection therefore uses only the pre-renaming prediction and does not condition on successful post-renaming behavior. We then ask whether the constraint-supported and residual components of those same annotations survive lexical removal.

4.7. Calibration and Project-Level Uncertainty

Stable-informative yield measures reproducibility and useful specificity, not semantic correctness. As a complementary calibration analysis, we compare four-run modal agreement with exact recovery of the developer annotation. The developer annotation is not assumed to be the only valid type; exact recovery instead provides an observable reference for asking whether repeated agreement carries information about the annotation recorded in the corpus.

The large number of annotation sites also creates a statistical dependence issue: sites from the same project should not be treated as independent replicates. We therefore use projects as the uncertainty unit. Our primary uncertainty analysis resamples the 36 source projects with replacement for 10,000 bootstrap replicates and recomputes the paper-facing contrasts.

For RQ1, in addition to the original concrete-minus-`none` gap, we report the difference-in-differences

$$\mathrm{DID} = [\mathrm{SI}_{\mathrm{orig}}(\texttt{none}) - \mathrm{SI}_{\mathrm{ren}}(\texttt{none})] - [\mathrm{SI}_{\mathrm{orig}}(\mathrm{concrete}) - \mathrm{SI}_{\mathrm{ren}}(\mathrm{concrete})].$$

A positive value means that removing lexical identity reduces stable-informative yield more strongly where no hard constraint is detected. For RQ2, the primary contrast is

$$\mathrm{Loss}(\texttt{none}) - \mathrm{Loss}(\mathrm{concrete}),$$

conditioned on originally stable-informative predictions.

To characterize cross-model heterogeneity, we additionally fit project-clustered logistic models. For RQ1 we fit

$$\mathrm{StableInformative} \sim \mathrm{Constraint} \times \mathrm{Condition} \times \mathrm{Model},$$

and test the constraint-by-condition-by-model interaction. For RQ2 we fit

$$\mathrm{InformationLoss} \sim \mathrm{Constraint} \times \mathrm{Model},$$

and test the constraint-by-model interaction. Because these Wald tests rely on asymptotic cluster-robust inference with only 36 source projects, we treat them as secondary evidence about model heterogeneity and use project-bootstrap effect sizes and confidence intervals as the primary uncertainty summaries. We repeat the cross-model analysis on both model-specific and common-site populations.

5. Results

5.1. RQ1: Inferability Predicts Stabilization

Table 1 reports stable-informative yield for the two extreme inferability classes before and after structure-preserving renaming.

The direction is identical across all six models. On the original programs, fully constrained sites exceed sites with no detected hard type constraint by 14.18–37.64 percentage points in stable-informative yield. Thus, the relationship is not an artifact of one model family or one model's overall willingness to produce specific types.

Removing lexical identity amplifies the separation in every model. The concrete-minus-`none` gap rises to 29.49–67.90 points after renaming. More directly, the difference-in-differences is positive in all six models: iso-renaming reduces stable-informative yield 15.31–34.99 percentage points more at `none` sites than at concrete sites. This intervention-based contrast is stronger than an ordinary association with site difficulty: renaming affects every site, but its impact depends systematically on whether hard program constraints continue to anchor the requested information.

The `none` rates also vary substantially across models. On the original programs they range from 42.60% to 82.21%; after renaming they range from 8.40% to 64.92%. Hard constraints therefore provide a shared source of support without eliminating model-specific differences in how effectively the residual inference problem is solved.

5.1.1. Does repeated stability carry useful information?

Stability does not establish correctness, but it can still carry information about prediction quality. As an auxiliary calibration analysis, we compare four-run modal agreement with exact recovery of the developer annotation for GPT-5, DeepSeek, and Claude. Table 2 shows the result on the original programs.

Across the three agreement levels, exact recovery rises sharply with repeated agreement in every model. At unanimous four-run agreement, more than 81% of predictions exactly recover the developer annotation, compared with only 18.67%–32.82% at modal agreement 0.50. We therefore do not equate stability with correctness, but repeated agreement is strongly calibrated with recovery of the observed reference annotation. The RQ1 result thus concerns a behavior that is empirically informative rather than arbitrary repetition.

Finding 1. *Hard semantic inferability predicts useful stabilization across all six models. Removing lexical identity amplifies the concrete-versus-`none` separation: iso-*

Table 1. Stable-informative yield before and after structure-preserving identifier renaming. Δ_{orig} and Δ_{ren} are concrete-minus-`none` gaps. DID is the additional renaming-induced drop at `none` relative to concrete, computed from unrounded rates.

		Original program			Renamed program			
Model	Sites	Concrete	None	Δ_{orig}	Concrete	None	Δ_{ren}	DID
GPT-5	52,734	87.95%	53.32%	34.63	82.78%	23.43%	59.35	24.71
DeepSeek	52,734	92.15%	59.24%	32.91	83.68%	15.78%	67.90	34.99
Claude	52,734	96.56%	77.80%	18.75	92.29%	50.02%	42.27	23.52
GPT-6	52,734	96.39%	82.21%	14.18	94.41%	64.92%	29.49	15.31
Qwen3-Coder	42,997	77.36%	46.18%	31.18	63.82%	12.85%	50.97	19.79
Devstral 2	36,465	80.24%	42.60%	37.64	69.21%	8.40%	60.81	23.17

Table 2. Exact recovery of developer annotations at increasing levels of four-run modal agreement on the original programs.

	Modal agreement		
Model	0.50	0.75	1.00
GPT-5	27.79%	52.00%	81.62%
DeepSeek	18.67%	46.11%	82.22%
Claude	32.82%	45.24%	83.28%

Table 3. Total information loss after structure-preserving identifier renaming, conditioned on originally stable-informative predictions. Δ is `none` minus concrete.

Model	Concrete	None	Δ
GPT-5	2.46%	32.84%	30.38
DeepSeek	3.33%	35.68%	32.35
Claude	1.09%	15.15%	14.06
GPT-6	0.73%	6.02%	5.29
Qwen3-Coder	10.37%	32.98%	22.61
Devstral 2	3.33%	28.31%	24.97

renaming reduces stable-informative yield 15.31–34.99 percentage points more where no hard constraint is detected.

5.2. RQ2: Inferability Predicts Robustness to Lexical Removal

We next condition on sites that are stable-informative in the original program and ask what happens to that established information after renaming. Table 3 reports total information loss, including both partial and complete loss.

The corrected result is not that unconstrained predictions universally collapse. Rather, hard semantic support consistently changes the probability that established information survives lexical removal. Across all six models, `none` sites incur 5.29–32.35 percentage points more total information loss than concrete sites.

The complementary preservation rates show the same separation. Concrete predictions are preserved 29.82–67.68 percentage points more often than `none` predictions. Fragility also appears through instability rather than information loss alone: renamed predictions at `none` sites destabilize substantially more frequently than their concrete counterparts in every model. Thus, removing lexical identity can cause a weakly anchored prediction either to lose specificity or to cease agreeing across repeated generations.

Figure 1 provides one concrete instance. Both `hass` parameters initially receive the same stable `HomeAssistant` prediction from GPT-5. For `async_setup.hass`, the dependency-derived actual-to-formal relationship survives renaming, and the renamed type remains anchored. For `check_stream.hass`, no corresponding hard constraint exists; GPT-5 falls back to `Any`. Other models can nevertheless preserve that second prediction, consistent with weaker residual evidence that our hard constraint analysis does not prove.

5.2.1. Controlling for annotation-site composition

The concrete and `none` populations have markedly different parameter/return mixtures. Of the 21,518 concrete sites, 16,029 (74.49%) are returns, whereas only 3,497 of the 29,758 `none` sites (11.75%) are returns. This imbalance creates an important alternative explanation for the aggregate result: the apparent robustness advantage of concrete sites could arise simply because concrete sites contain many more returns.

We therefore repeat the RQ2 comparison separately within parameters and returns. Table 4 reports the `none`-minus-concrete total information-loss gap after conditioning, as in the main RQ2 analysis, on predictions that were already stable-informative in the original program.

The result is consistent across both annotation kinds. Within parameters, `none` sites incur 5.18–25.46 percentage points more total information loss than concrete sites; within returns, the gap is 5.84–27.43 points. All 12 model-by-site-kind contrasts have the predicted positive sign. For GPT-5, DeepSeek, Claude, and GPT-6, the parameter and return gaps are also similar in magnitude, while Qwen3-Coder and Devstral 2 show a larger effect for returns.

This stratification substantially strengthens the robustness

Table 4. Total information-loss gap after stratifying by annotation kind. Values report the `none`-minus-concrete difference among predictions that were stable-informative on the original program, in percentage points.

Model	Parameters	Returns
GPT-5	25.46	27.43
DeepSeek	24.76	26.58
Claude	12.11	14.44
GPT-6	5.18	5.84
Qwen3-Coder	10.43	22.71
Devstral 2	19.14	25.53

Table 5. Original-to-renamed preservation of constraint-supported and residual components at shape-only sites whose original modal prediction is semantically consistent with the inferred constraint. Supported and residual rates use the same paired-site denominator.

Model	Pairs	Supported	Residual	Gap
GPT-5	443	99.32%	65.24%	34.09
DeepSeek	478	99.58%	69.04%	30.54
Claude	526	97.34%	73.57%	23.76
GPT-6	495	99.39%	76.36%	23.03
Qwen3-Coder	228	100.00%	57.89%	42.11
Devstral 2	247	100.00%	56.28%	43.72

result. Because RQ2 already conditions on predictions that had stabilized before renaming, the comparison asks whether established information survives lexical removal *within the same annotation kind*. The answer is consistently yes: hard semantic support is associated with greater preservation for both parameters and returns. The aggregate RQ2 effect therefore cannot be explained by the concrete class containing more returns or the `none` class containing more parameters.

For completeness, the original-program RQ1 stabilization effect also remains positive within both kinds in every model: concrete-minus-`none` stable-informative gaps range from 6.68–27.10 percentage points for parameters and 19.86–43.15 points for returns.

5.2.2. A WITHIN-ANNOTATION TEST

Shape-only sites allow a stronger control: rather than comparing different sites, we compare the constraint-supported and unresolved components of the *same* prediction.

For the primary analysis, we select shape-only sites whose original modal prediction is semantically consistent with the structural constraint and then observe both components after renaming. Selection therefore depends only on the pre-intervention prediction and does not condition on successful post-renaming behavior.

Across all six models, the constraint-supported component is preserved in 97.34–100.00% of paired annotations, whereas the residual component is preserved in only 56.28–76.36%. The resulting within-annotation gaps range from 23.03 to 43.72 percentage points.

The same qualitative result holds without the original semantic-consistency filter: at shape-only sites, supported-component preservation ranges from 93.36–99.17%, compared with 55.69–76.41% for residual components.

This paired result provides direct evidence for the proposed mechanism. Renaming does not make an annotation uniformly harder. The part fixed by hard program semantics forms a robust core, while the unresolved component remains much more sensitive to weaker structure, lexical/API identity, and learned priors.

5.2.3. PROJECT-LEVEL ROBUSTNESS.

Because annotation sites are clustered within repositories, we resample the 36 source projects for 10,000 project-bootstrap replicates.

For RQ1, the difference-in-differences remains positive for every model. The 95% project-bootstrap intervals have lower bounds between 10.21 and 27.86 percentage points. For RQ2, the `none`-minus-concrete information-loss gap likewise remains positive for every model, with lower bounds between 3.38 and 26.63 points.

The direction also recurs across individual repositories rather than being driven solely by a few large projects. In the original-program RQ1 comparison, 88.57–97.22% of eligible projects exhibit the predicted positive concrete-minus-`none` gap. For RQ2, 69.44–91.67% exhibit greater information loss at `none` than concrete sites. The latter is less uniform across individual repositories, especially for GPT-6, but the pooled project-bootstrap effect remains strictly positive in all six models.

Finding 2. *Hard semantic support makes established predictions more robust to lexical removal. Among originally stable-informative predictions, the* `none`*-minus-concrete information-loss gap is positive in all six models separately for both parameters and returns. The relationship also survives project-level resampling and appears within individual annotations, where constraint-supported components are 23.03–43.72 percentage points more likely to survive renaming than residual components.*

5.3. RQ3: Shared Direction, Model-Dependent Magnitude

RQ1 and RQ2 establish the same qualitative relationship in every model, but they do not imply that model families respond identically. We therefore ask whether the *magnitude* of the inferability effect differs across models and whether those differences remain when the models are compared on the same sites.

Table 6 reports the two intervention-based effect sizes on both the model-specific populations and the all-model common-site populations.

Two conclusions follow. First, the *direction* is shared. Every RQ1 DID and every RQ2 loss gap is positive for all six models, both on each model's own usable population and after restricting the comparison to common sites. The close agreement between the two population definitions shows that the cross-model conclusions are not an artifact of different model-specific site coverage.

Second, the *magnitude* is not shared. Project-clustered interaction tests reject a model-invariant effect size for both analyses: the constraint-by-condition-by-model interaction for RQ1 and the constraint-by-model interaction for RQ2 are both significant ($p < 10^{-4}$). We treat these asymptotic tests as secondary to the project-bootstrap effect sizes, but the same heterogeneity appears there. For RQ1, DeepSeek's DID exceeds GPT-5's by 10.28 points with a 95% project-bootstrap interval of [3.20, 15.11], whereas GPT-6's is 9.40 points smaller with an interval of [-15.91, -4.84]. For RQ2, Claude and GPT-6 exhibit much smaller `none`-site penalties than GPT-5, by 16.32 and 25.09 points, respectively, with intervals excluding zero. The common-site analysis yields the same qualitative conclusions.

This combination—shared direction but heterogeneous magnitude—clarifies the role of semantic inferability. Hard constraints provide a common model-independent source of support, but they do not determine how effectively a model solves the remaining problem once such support is absent. GPT-6, for example, still benefits from concrete semantic support, yet its additional information-loss penalty at `none` sites is only 5.29 points, compared with more than 30 points for GPT-5 and DeepSeek. Semantic inferability is therefore not simply a scalar measure of generic task difficulty; it separates an input-determined core from a residual inference problem whose treatment depends on model capability and learned priors.

The identity of failures reinforces this interpretation. On the 5,016 `none` sites for which all six models make an originally stable-informative prediction, complete-loss events overlap only partially. A complete loss occurs in at least two models at 35.96% of sites and in at least three at 24.94%, but only 0.60% undergo complete loss in all six models. Pairwise Jaccard overlap among complete-loss sets ranges from 7.45% to 61.06%.

Thus, absence of a hard constraint does not induce one universal failure mode. Instead, it exposes a residual problem on which models can exploit different sources of weaker structural evidence and learned prior information. Hard semantic support produces *constraint-induced robustness*; where that support ends, we observe *residual-evidence divergence*.

Finding 3. *The inferability effect has the same direction across all six model families but differs substantially in magnitude. Hard constraints provide a shared robustness anchor; when they leave a residual inference problem, both the rate and identity of failures become markedly more model dependent.*

5.4. Beyond Hard Constraints: Residual Structural Evidence

The results do not support a binary distinction between "semantic reasoning" and "lexical guessing." Many `none` predictions remain robust after renaming even though our analysis establishes no hard type constraint, and the cross-model results show that different models can succeed on different subsets of these sites.

These cases suggest that LLMs can exploit weaker renaming-invariant relationships that fall below the threshold of a static proof. Local data flow, call shape, constructor-like correspondence, attribute use, and other structural patterns can narrow the completion space without uniquely determining a nominal type.

We examine this effect for `none` return sites. Approximately half of the matched raw population contains at least one call-valued return, and a subset directly returns expressions of the form

```
return A(...).
```

Such correspondence does not prove that `A` denotes the required type, but it creates a renaming-invariant structural relationship that an LLM may exploit. Direct-name correspondence is enriched among robust cases, although it explains only a minority of them, indicating that no single syntactic pattern accounts for the residual robustness.

Targeted manual inspection leads to the same conclusion. Loss-prone cases tend to lack sufficient surviving structural information to identify a specific type, whereas robust `none` sites more often retain weaker relational evidence not captured by the conservative hard-constraint analysis. The partial overlap of failures across models further suggests that model families exploit this residual evidence with different effectiveness.

Together, these observations suggest an evidence hierarchy, which we call *constraint-guided completion*: hard semantic constraints anchor the prediction and restrict its possible completions; weaker renaming-invariant structural evidence further narrows that space; and lexical/API cues and learned priors supply residual specificity where the program leaves alternatives open.

This account explains why stability and robustness are re-

Table 6. Cross-model effect sizes on model-specific and common-site populations. RQ1 reports the additional renaming-induced stable-informative drop at `none` relative to concrete. RQ2 reports the `none`-minus-concrete total information-loss gap.

	RQ1 DID		RQ2 loss gap	
Model	Model-specific	Common	Model-specific	Common
GPT-5	24.71	26.10	30.38	27.85
DeepSeek	34.99	34.78	32.35	31.19
Claude	23.52	24.35	14.06	9.91
GPT-6	15.31	16.43	5.29	4.80
Qwen3-Coder	19.79	23.03	22.61	22.01
Devstral 2	23.17	24.46	24.97	28.84

lated but not equivalent. A strong lexical cue can make a weakly constrained prediction perfectly stable across repeated generations, as in the original `hass` example, while that stability can disappear after the cue is removed. Conversely, sufficiently strong residual structural evidence can allow another model to preserve the same prediction even without a detected hard constraint.

6. Related Work

LLM stability and uncertainty. Prior work documents nondeterminism across repeated LLM generations and sensitivity to prompt perturbations (Ouyang et al., 2025; Atil et al., 2024; Sclar et al., 2024). Repeated sampling and semantic agreement have also been used to characterize uncertainty and improve reasoning reliability (Wang et al., 2023; Kuhn et al., 2023). These approaches primarily measure model behavior after generation. Our work asks whether an independent input-side property—how strongly the available information constrains a prediction—explains where stable behavior emerges.

Recent work further argues that scalar outcome measures can obscure structure in the reasoning process. Jiang et al. introduce TRACED, which characterizes reasoning trajectories using geometric notions of progress and stability (**?**). This perspective is complementary to ours: trajectory-based approaches characterize structure in the model's reasoning process, whereas semantic inferability characterizes structure in the problem presented to the model.

Attributing sources of model information. Morris et al. distinguish information attributable to memorization from information attributable to generalization, emphasizing that an observed model output does not by itself identify the source of the information behind it (Morris et al., 2026). Our attribution question is orthogonal and occurs at inference time: which information in a prediction is already supported by the current input, and which specificity must be supplied from residual evidence?

Learned type inference and structural code models. Statistical and neural type-inference systems combine lexical, syntactic, and program-structural information (Xu et al., 2016; Wei et al., 2020; Allamanis et al., 2020; Wei et al., 2023; Peng et al., 2023). Structure-aware code representations similarly show the value of relational and data-flow information beyond token sequences (Hellendoorn et al., 2020; Guo et al., 2021; Zhang et al., 2024). Our goal differs from improving annotation accuracy: we use program-derived typing constraints as an explanatory variable for LLM stability and robustness.

7. Limitations and Broader Scope

Our experiments establish the relationship between semantic inferability and LLM behavior in Python type inference. We do not claim that the present experiments establish the same relationship for arbitrary reasoning tasks. Type inference provides a useful setting precisely because input-side constraints can be characterized explicitly at individual prediction sites and lexical identity can be manipulated while supported program relationships are preserved.

The underlying distinction, however, is not specific to typing. Many reasoning tasks differ in how strongly the available information constrains the answer: premises constrain conclusions, specifications constrain program behavior, observations constrain diagnoses, and requirements constrain valid plans. Our results therefore motivate a broader hypothesis that model behavior should be interpreted relative to the inferability of the requested information. Establishing whether analogous operationalizations predict LLM robustness in other domains is future work.

Our inferability analysis is conservative rather than complete. Python's dynamic features and incomplete dependency information make exhaustive recovery of all semantically relevant type information infeasible. The four inferability classes should therefore be interpreted operationally as the information established by a specified analysis. In particular, `none` does not imply absence of all useful evidence. The substantial number of robust `none` predictions, and their model-dependent behavior, demonstrate why this

qualification matters.

Relatedly, inferability should not be interpreted as a complete model-independent difficulty score. All six models benefit from hard semantic support, but the size of that benefit differs significantly across model families. The residual inference problem can therefore remain easy for one model and difficult for another even when the hard-constraint classification is the same.

Structure-preserving renaming is likewise an intervention rather than perfect isolation of a single causal factor. It removes project-specific lexical and API identities while preserving supported program relationships, but can also alter learned associations that interact with those identities. We therefore interpret information loss as dependence on evidence removed by the transformation, rather than claiming that every lost type was inferred solely from its identifier.

Finally, stability is not correctness. A model may repeatedly produce the same incorrect annotation, and a developer annotation need not be the only semantically valid type. Stable-informative yield therefore measures whether useful specificity stabilizes rather than semantic correctness. Nevertheless, our calibration analysis shows that stronger four-run agreement is associated with substantially higher exact recovery of developer annotations.

8. Conclusion

What makes an LLM prediction stable? In Python type inference, a substantial part of the answer is how strongly the input itself constrains what can be generated. Across six model families, fully constrained sites produce more stable-informative predictions than sites with no detected hard constraint, and structure-preserving lexical removal amplifies that separation. The renaming-induced stable-informative drop is 15.31–34.99 percentage points larger at `none` sites, and among predictions that have already stabilized, `none` sites incur 5.29–32.35 points more information loss. Both effects remain positive under project-level resampling.

The same distinction appears inside individual annotations. At shape-only sites, constraint-supported components survive lexical removal in 97.34–100.00% of paired cases, compared with only 56.28–76.36% for components left unresolved by those constraints. Hard semantic information therefore behaves as a robust core rather than simply correlating with an easier class of sites.

At the same time, the magnitude and identity of residual failures differ across model families. This yields a more nuanced picture than either "semantic reasoning" or "lexical guessing." Hard constraints create a shared robustness anchor; weaker structural relationships narrow the remaining candidate space; and model-specific capabilities, lexical/API cues, and learned priors complete what remains unresolved.

Semantic inferability consequently provides an input-side explanation for three aspects of LLM behavior that are often studied separately: when useful predictions stabilize, when they survive meaning-preserving perturbations, and where model families begin to diverge. More broadly, the results suggest that understanding LLM reliability requires characterizing not only model outputs but also how much of each requested answer is already determined by the problem presented to the model.

References

Allamanis, M., Barr, E. T., Ducousso, S., and Gao, Z. Typilus: neural type hints. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI 2020, pp. 91–105, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450376136. doi: 10.1145/3385412.3385997. URL https://doi.org/10.1145/3385412.3385997.

Atil, B., Chittams, A., Fu, L., Ture, F., Xu, L., and Baldwin, B. Llm stability: A detailed analysis with some surprises. *arXiv preprint arXiv:2408.04667*, 1, 2024.

Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., LIU, S., Zhou, L., Duan, N., Svyatkovskiy, A., Fu, S., Tufano, M., Deng, S. K., Clement, C., Drain, D., Sundaresan, N., Yin, J., Jiang, D., and Zhou, M. GraphCodeBERT: Pre-training code representations with data flow. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=jLoC4ez43PZ.

Hellendoorn, V. J., Sutton, C., Singh, R., Maniatis, P., and Bieber, D. Global relational models of source code. In *International Conference on Learning Representations*, 2020. URL https://openreview.net/forum?id=B1lnbRNtwr.

Kuhn, L., Gal, Y., and Farquhar, S. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=VD-AYtP0dve.

Mir, A. M., Latoskinas, E., and Gousios, G. Manytypes4py: A benchmark python dataset for machine learning-based type inference. In *IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*, pp. 585–589. IEEE Computer Society, May 2021. doi: 10.1109/MSR52588.2021.00079.

Morris, J. X., Sitawarin, C., Guo, C., Kokhlikyan, N., Suh, G. E., Rush, A. M., Chaudhuri, K., and Mahloujifar, S. How much can language models memorize? In *Forty-third International Conference on Machine Learning*, 2026. URL https://openreview.net/forum?id=bA6BgSbaUi.

Ouyang, S., Zhang, J. M., Harman, M., and Wang, M. An empirical study of the non-determinism of chatgpt in code generation. *ACM Transactions on Software Engineering and Methodology*, 34(2):1–28, 2025.

Peng, Y., Wang, C., Wang, W., Gao, C., and Lyu, M. R. Generative type inference for python. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 988–999, 2023. doi: 10.1109/ASE56229.2023.00031.

Sclar, M., Choi, Y., Tsvetkov, Y., and Suhr, A. Quantifying language models' sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting. In *International Conference on Learning Representations*, volume 2024, pp. 25055–25083, 2024.

Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A., and Zhou, D. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=1PL1NIMMrw.

Wei, J., Goyal, M., Durrett, G., and Dillig, I. LambdaNet: Probabilistic type inference using graph neural networks. In *International Conference on Learning Representations*, 2020. URL https://openreview.net/forum?id=Hkx6hANtwH.

Wei, J., Durrett, G., and Dillig, I. TypeT5: Seq2seq type inference using static analysis. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=4TyNEhI2GdN.

Xu, Z., Zhang, X., Chen, L., Pei, K., and Xu, B. Python probabilistic type inference with natural language support. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, FSE 2016, pp. 607–618, New York, NY, USA, 2016. Association for Computing Machinery. ISBN 9781450342186. doi: 10.1145/2950290.2950343. URL https://doi.org/10.1145/2950290.2950343.

Zhang, D., Ahmad, W. U., Tan, M., Ding, H., Nallapati, R., Roth, D., Ma, X., and Xiang, B. CODE REPRESENTATION LEARNING AT SCALE. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=vfzRRjumpX.